



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)

LAB REPORT NO : 03

Course Title: Operating System Lab

Course Code: CSE-402 Section: 231-D2

Student Details

Name		ID
1.	Promod Chandra Das	231002005

Lab Date : 25-11-25

Submission Date: 02-12-25

Course Teacher's Name : Md. Shoab Alam

[For Teachers use only: **Don't Write Anything inside this box**]

<u>Lab Report Status</u>	
Marks:	Signature:
Comments:	Date:

1. TITLE OF THE LAB EXPERIMENT :

Implementation of Round Robin Algorithm.

2. OBJECTIVES :

- 1.To understand the working principle of Round Robin (RR) CPU scheduling algorithm.
2. To implement Round Robin scheduling using a shell script.
3. To calculate Waiting Time (WT) and Turnaround Time (TAT) for each process.
4. To observe the effect of time quantum on CPU performance.
- 5.To compare RR with other scheduling algorithms based on fairness and response time.

3. PROCEDURE :

1. Start by taking:
 - Number of processes
 - Burst time of each process
 - Time Quantum
- 2.Initialize:
 - Remaining burst time array
 - Waiting time = 0 for all
 - Turnaround time = 0 for all
- 3.Execute processes in a circular order:
 - If remaining burst time $\geq$ quantum $\rightarrow$ execute for quantum
 - Else $\rightarrow$ execute remaining time
- 4.Keep a running clock (time counter).
- 5.When a process finishes, record:
 - Turnaround Time = finish time
 - Waiting Time = turnaround time – burst time
- 6.Continue until all processes finish.
- 7.Display Gantt chart, WT, TAT, and average times.

4. IMPLEMENTATION :

(Shell Script: Round Robin Scheduling)

```
#!/bin/bash

echo -n "Enter number of processes: "
read n

echo "Enter burst times:"
for ((i=0;i<n;i++))
do
    echo -n "Burst time for P$((i+1)): "
    read bt[$i]
    rt[$i]=${bt[$i]}
done

echo -n "Enter Time Quantum: "
read tq

time=0
count=$n

while [ $count -ne 0 ]
do
    for ((i=0;i<n;i++))
    do
        if [ ${rt[$i]} -gt 0 ]
        then
            if [ ${rt[$i]} -le $tq ]
            then
                time=$((time + ${rt[$i]}))
                tat[$i]=$time
                rt[$i]=0
                count=$((count - 1))
            else
                rt[$i]=$(( ${rt[$i]} - tq ))
                time=$((time + tq))
            fi
        fi
    done
done

echo -e "\nProcess\tBT\tTAT\tWT"
totalWT=0
totalTAT=0

for ((i=0;i<n;i++))
do
    wt[$i]=$(( ${tat[$i]} - ${bt[$i]} ))
    echo -e "P$((i+1))\t${bt[$i]}\t${tat[$i]}\t${wt[$i]}"
    totalWT=$((totalWT + ${wt[$i]}))
    totalTAT=$((totalTAT + ${tat[$i]}))
done
```

```
echo -e "\nAverage Waiting Time = $(echo "scale=2; $totalWT / $n" | bc)"
echo "Average Turnaround Time = $(echo "scale=2; $totalTAT / $n" | bc)"
```

5. Output :

```
See "man sudo_root" for details.

promod@Ubuntu24:~$ touch lab3.sh
promod@Ubuntu24:~$ chmod +x lab3.sh
promod@Ubuntu24:~$ ./lab3.sh
Enter number of processes: 3
Enter burst times:
Burst time for P1: 7
Burst time for P2: 4
Burst time for P3: 9
Enter Time Quantum: 3

Process BT      TAT      WT
P1      7       17      10
P2      4       13       9
P3      9       20      11

Average Waiting Time = 10.00
Average Turnaround Time = 16.66
promod@Ubuntu24:~$
```

6. ANALYSIS AND DISCUSSION

1. Fairness:

Round Robin is one of the fairest scheduling algorithms because each process gets an equal share of CPU time in cyclic order.

2. Effect of Time Quantum:

- If **quantum is too small**, context switching increases → overhead becomes high.
- If **quantum is too large**, RR behaves like FCFS → response time increases.

3. Performance:

RR ensures **good response time**, making it suitable for:

- Time-sharing systems
- Interactive systems

4. Observation from Output:

- Processes with smaller burst time finish earlier.
- Larger burst time processes receive CPU multiple times.
- Waiting time depends heavily on the quantum value.

5. Practical Uses:

RR is widely used in multitasking OS like Linux/Unix for process scheduling.

6. SUMMARY

Round Robin CPU Scheduling Algorithm is a pre-emptive, fair, and widely used scheduling method. It assigns each process a fixed time slice (quantum) and cycles through all processes until completion. In this experiment, we implemented RR using a shell script, calculated WT and TAT, and observed how quantum affects system performance. RR is most effective in time-sharing environments where responsiveness and fairness are critical.

